STAT 587: Preliminary Draft of Group 5's Final Project

Due on March 04th

Trang Nguyen and Takeshi Stormer

March 4, 2026

Final Report Requirements

1 Introduction

Motivation Beyond the pursuit of generating infinite wealth by predicting the future values of the stock market (or inadvertently collapsing any concept of having a financial market to begin with), simply being content with being able to predict the direction of the market (emphasis on the US stock market) of the following day allows for individuals to trade and/or invest more wisely. The motivation behind understanding this problem, and subsequently attempting to solve it, is given by the consequences it could bring to the financial industry. The conclusion of this paper will give insight as to whether or not the aforementioned data is adequate towards predicting the direction of the US stock market, and if not, gives rise to the question of whether or not it is possible to find any amount of data that will.

Domain context: Financial Economics Our model with different window of time may answer the question how efficient the market is. That is, if the prediction using the hourly data is better than they daily data, we can say that the market efficiency is much better today.

1.1 Research problem

Our project aim at predicting the Direction of the S&P500 Index. More specifically, we're interested in determining the direction of the market on the following day (on a day-to-day basis).

1.2 Background context

1.3 Dataset description

Data include price metrics and volumn traded of stocks that are part of the S&P 500 index spanning January 1st, 2024, to December 31st, 2025. It needs to be noted that holidays in the US, Germany and Japan do not necessarily coincide. Front-filling was applied to indexes to account for these holidays. We removed rows based on whether or not a holiday was observed by all stocks (in other words, all values in one row for stocks were NA). The resulting preprocessed dataset should contain no NAs. Some further cleaning was performed to remove stocks that were listed on a date past January 1st.

-

- Variables:
  - Base features (raw data dowloaded):
    * Open Price (OP)
    * Close Price (CP)
    * High Price (HP)
    * Low Price (LP)
    * Traded Volume (TV)
    * Day of the week
    * Month of the year
  - Generated Features (for Price metrics):
    * Lag (over varying periods). Due to the nature of time series data, the lag values are important in predicting the direction of the market next day. Except for the setting of the neural network methods to include each data points in the model, all other methods for prediction requires additional predictors of lag values to be created before the training process. The lag period is 21 days for some methods and 7 days for the others due to computationally intensive nature of the latter and our group's time and resource constraints
    * OP, CP, HP, LP Percent Change (PC)
    * Exponential Moving Average (over varying windows)
    * Volatility (over varying periods)
    * Max/Min Channel Position (over varying windows)
    * Rolling Volume Weighted Average Prices (VWAP) (over varying windows)
    * Rolling Z-Scores (over varying windows)
    * Daily Range
    * If time permits, we will attempt to add average stock price and standard deviation of a stock price within a day, and day of week analysis.
  - Note on Generated Features:
    * Percent change is utilized to have stationary data. Without this, we can run into issues where a method involving decision trees utilize outdated prices that are not relevant to future prices (i.e. the value of a share goes from \$10 to \$100, an internal node with condition [Close $<$ \$25] will never be activated if the relevant price as of present is \$100)
    * Lag captures momentum, which will allow us to see if past data has any impact on future values.
    * Volatility looks to capture how much a stock can be expected to move within a certain day. For the extent of this project, volatility can be seen as the standard deviation of price movements.
    * The exponential moving average captures more recent price movements quicker than a normal moving average by recursively generating a new average with a weight that emphasizes the new price point. This allows for an increased ability to analyze short-term trends as we are looking to predict the following day while maintaining the benefits of a moving average.

    * A Max/Min channel position allows for identification of support and resistant lines, which can further provide insight of potential breakouts.
    * The rolling volume weighted average price looks to portray the value of share based on the amount people are willing to pay for it. This ensures that price points where investors are primarily buying or selling shares at is reflected, as opposed to seeing potentially a misleading share price.
    * The rolling z-score utilizes the exponential moving average and volatility in order to generate z-scores of a price movement. It is good to note that a t-distribution will probably fit this projects use-case, as there are days influenced by external events that cause price movements you would be believe to not occur given a z-score.
    * **IMPORTANT**: Features involving non-stationary metrics (EMA, OP, CP, HP, LP, VWAP) will be removed when K-fold cross-validation is introduced.

- Size: The raw data contains 501 rows, and 2520 columns (roughly around 20 megabytes worth of data). The sizable number of columns is explained by each stock (503) and index (1) having their own price metrics for CP, OP, HP, LP and TV ([503+1]$*$5) = 2520. The number of rows is explained by the exclusion of 19 holidays and 211 weekend days.

Key points: Large number of features, higher than number of observation. Data is time series

1.4 Exploratory Data Analysis

The following heatmaps are created utilizing the engineered feature 'Close PC' and the volatility time-series is created utilizing the engineered feature 'Close VOL 28'.

- Correlation Heatmap (All Sectors):

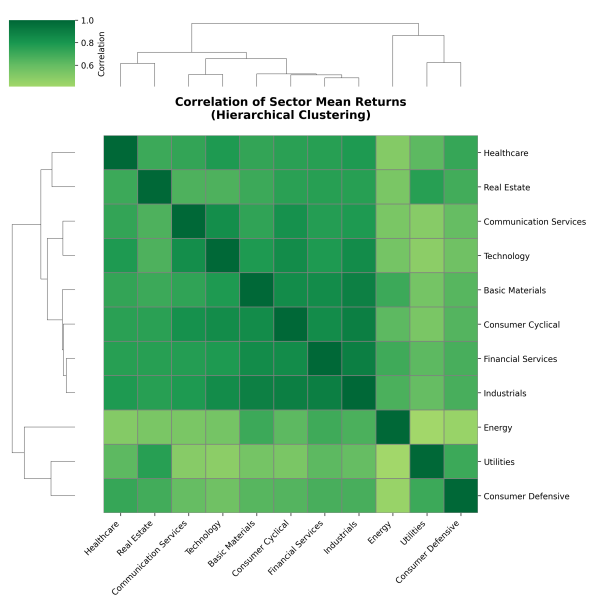

Figure 1: Sector Heatmap (All)

- Correlation Heatmap (Technology)

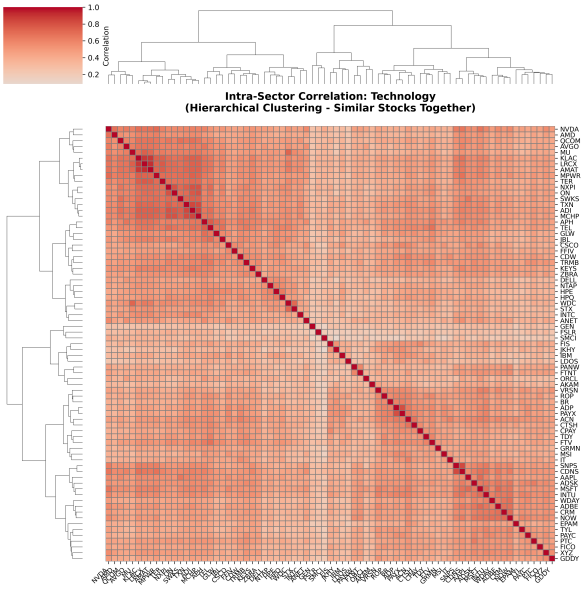

Figure 2: Sector Heatmap (Technology)

- Correlation Heatmap (Industrial)

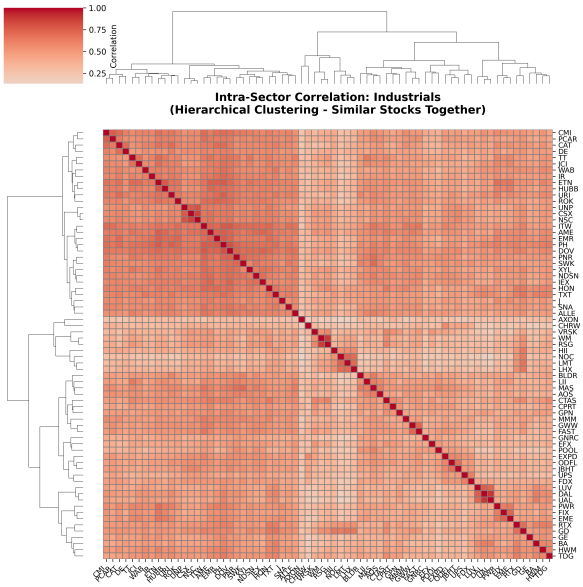

Figure 3: Sector Heatmap (Industrial)

- Correlation Heatmap (Financial Services)

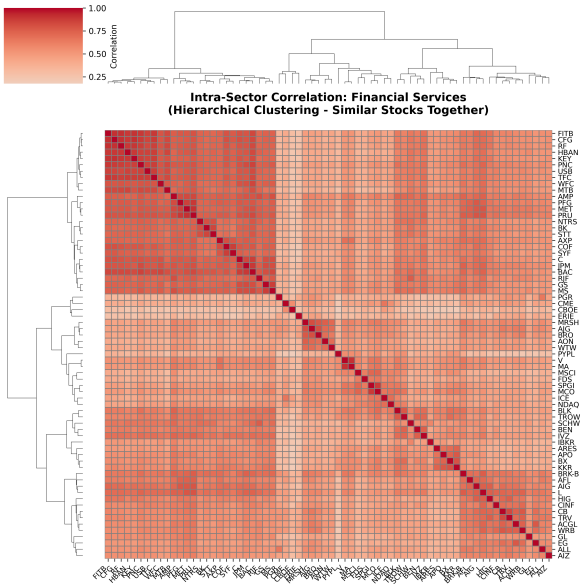

Figure 4: Sector Heatmap (Financial Services)

- Volatility Time-Series of All Sectors.

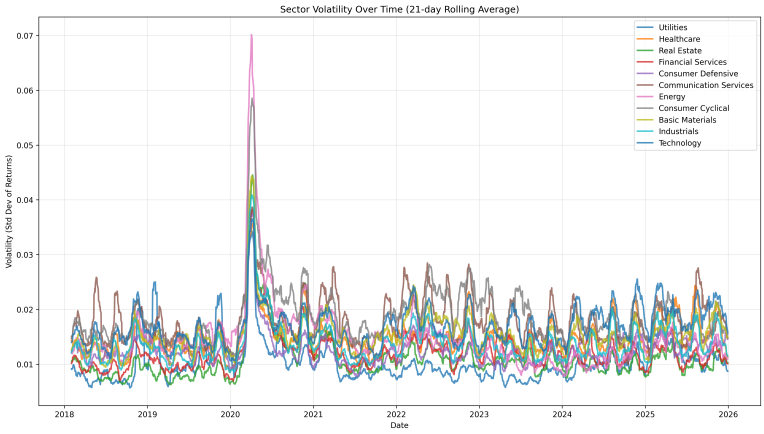

Figure 5: Sector Volatility (All)

- Volatility Time-Series of All Sectors (Smoothed)

Figure 6: Sector Volatility (All) (Smoothed)

2 Methods

As our data set is time series and has a large number of features compared to the number of data points, some methods prove to have more merits, namely random forest tree classification, support vector machine, and neural networks (CNN, simple RNN, LSTM). These methods, however, require a certain level of computational capacity, which is not very extensive giving our constraints of time and resources. At the same time, basic generalized linear model like logistic regression is still applicable with additional dimensional reduction technique (Ridge, LASSO, and PCA). We have not concluded which are 3 methods should be focus in this project. Instead, we provided all preliminary result and seek for further advices.

Logistic Regression with dimensional reduction or feature selections is our simplest model to be implemented. This method would result in a predicted response that measures the probability of the following day being a positive day. As it requires that the number of predictors smaller than the number of data points, we have to apply some dimensional reduction techniques, namely Ridge and LASSO regression, along with Principal Component Analysis. The necessity to use these techniques is further emphasized by the fact that there are systematic market movement making data of different stocks highly correlated. Also, multiple features are secondary indicators constructed from the raw data, resulting in the same issue of multicollinearity. Though the loss of interpretability could discourage us from using PCA and Ridge regression, our primary focus is on the prediction of the market direction is for their use. In almost all cases, there was data scaling (via StandardScaler provided by scikit-learn) performed when doing dimensionality reduction.

With the available logistic regression method provided by the library scikit-learn, LogisticRegressionCV and LogisticRegression, a penalty term was inherently already a part of initializing the model. However, there was still another motivation for us to reduce the amount of features (columns) before fitting logistic regression, since it would otherwise take an extended period of time to run (especially since convergence would be impossible with stocks or engineered features that experienced a great deal of multicollinearity); PCA, Ridge, and LASSO method are used in this feature selection/engineering. As the number of features is very large, we do not use traditional method of forward-stepwise/ backward-stepwise/ forward-stagewise/ best subset selection.

The second method experimented by our group is Random Forest Tree Classifier. This method would directly forecast the direction and is not restricted by the requirement of $p < n$ like in the generalized regression model. This allows us to fit the model with all features, and through feature-importance, deduce which features carry the biggest weight. Because of a large number of predictors, we do not start with a base tree classifier. Random Forest model has a merit in this case as it allow us build a tree with a cap of feature number.

There was some initial exploration into utilizing a random forest regressor then converting it's predictions into a binary response by checking for a positive or negative prediction. However, due to time-constraints, the exploration was stopped and only the random forest classifier was continued with. In this case, though the random forest classifier did not enforce that the number of parameters be less than that of the number of observations, there is no indication in the code that it was explored as a viable model. Instead, other dimensionality reduction methods were applied standalone (no combination of such methods), including: PCA, Ridge

regression, LASSO regression, and Step-wise Regression utilizing walk forward validation.

The third method employed is Support Vector Machine. Similar to Random Forest method, it does not require $p < n$. Further more, SVM is a straight forward solution to a binary classification problem with more features than data points as it use kernel to capture data point proximity. We started our SVM model with a linear kernel as large feature number but finally choose RBF kernel due to its better performance.

Additional method employed in our analysis is neural network with the focus on convolutional, simple recurrent, and LSTM ones (7 days window). As the neural network learning algorithm already use all data points in constructing each nodes, it is not necessary to use lag values as predictors in these models. Among CNN, simple RNN, and LSTM, the most natural option among them is the last one because its nature is very close to the market reaction in time series data. The other networks are included partly because of our concern about LSTM computing time and partly due to our curiosity of their application for time series. More specifically, 2D-CNN is possible for a time relationship between data points while a simple RNN model can reflect somehow the relationship of market movement at different times. Overall, the neural network method are feasible when we extended the time period for the data loaded. This method, however, is a black box and is not interpretable.

Regarding to the selection of optimal parameters for the above mentioned methods, standard cross-validation are used for all models while walk-forward validation are employed in some feasible ones. For the models that is computationally intensive, the number of fold is set to be 5 while our previous milestone indicate 10-fold cross validation. The general selection of hyperparameter is based on the model with highest average train accuracy/ F1 score while attain an acceptable level of standard deviation. Detailed of optimal hyperparameter for each model is in included in the appendix.

Further part need to be addressed: Baseline model is RF, Logistic, SVM with indexes only. • Mathematical formulation (optional but encouraged) • Why methods are appropriate • Hyperparameter choices • Python implementation details • Textbook citation: "See ESL Chapter X, Section Y"

3 Results

3.1 Model Comparison

Table 1: Logistic Regression Comparison: Base L2 vs Elastic Net vs Feature-Engineered L2 (24-hour lag features)

Metric / Parameter	**L2 (Ridge)**	**Elastic Net**	**FeatureEng L2**
Accuracy	0.5820	0.5841	0.5181
Precision	0.5877	0.5877	0.5375
ROC-AUC	0.6115	0.5975	0.5141
Algorithm	Logistic Regression (L2)	Logistic Regression (Elastic Net)	Logistic Regression (L2
Penalty	l2	elasticnet	l2
Solver	lbfgs	saga	lbfgs
Max iterations	1000	2000	1000
Cross-validation	5-fold stratified	5-fold stratified	5-fold stratified
Optimal C	166.810054	166.810054	0.000100
$l1_ratio$	N/A	0.90	N/A

3.2 Visualization of predictions

3.3 Validation results

3.4 Overfitting/underfitting analysis

4 Discussion

- Interpretation of findings
  - Real-world implications
  - Limitations: time constraint, resource constraint for computation
  - Possible methodological extensions

5 Reproducibility Requirements

- Environment file or package list
  - Folder structure
  - Random seed control
  - Clear function/code organization

Additional comments: more features vs just basic: - ridge reduces coefficient variance, not the redundancy in the data. - Curse of dimensionality / weak signals: extra features increase complexity n, so the best achievable generalization can decrease.

Python Notebook

- Project Repository: `https://github.com/Sabuchan123/STAT-587-Final-Project`
- LaTeX folder: https://www.overleaf.com/read/bpdvnbmwzqhc#6b670c